
mygit + RAG

30-Day Build Plan

Week	Focus	Key output
1 (Days 1–7)	TypeScript + git object model	utils.ts + object.ts complete
2 (Days 8–14)	Store, Index, Refs, Repository	Full git CLI working
3 (Days 15–21)	Networking + Backend API	Push/pull/clone + deployed API
4 (Days 22–28)	Frontend + RAG pipeline	UI live + Ask tab working
Final (Days 29–30)	Polish + demo	README + demo video

“Close the reference code before you type. Read a file, understand every line, then write it from memory. The goal is a mental model you can explain in any interview — not just a working program.”

Time budget: 1.5 hrs/weekday evening + 5 hrs/weekend day = $\approx$ 80 hrs total • TypeScript • Node.js • PostgreSQL • React • OpenAI

How to use this plan

Tick the checkbox when the day is done — not when you *started* it. A day is done when the test at the bottom of the card passes.

If you fall behind: days are sequenced, not dated. If Day 11 takes two evenings, spend two evenings. Don't skip the test to move on.

Weekend days (starred *) have longer blocks and cover more. Plan project work for 9 AM – 1 PM, not “later.”

TypeScript: spend 2 hours on Day 1 reading the handbook basics, then learn the rest by building. By Day 7 it will feel natural.

Pro Git — Chapter 10 Reading Guide

Read alongside your project, not all at once before you start. Day 3 is the core reading session. The others slot in naturally as you reach each phase.

READ Read every word and run every command in your terminal. **SKIM** One pass — understand the concept, not the detail. **SKIP** Not needed for this project.

READ 10.1 — Plumbing and Porcelain

Day 3 · 30 min

Sets up the mental model for the whole chapter. Git has two layers: *porcelain* (the commands you use: `commit`, `checkout`) on top of *plumbing* (the low-level primitives underneath: `hash-object`, `cat-file`, `update-index`). You are building the plumbing. This context makes the rest of the chapter make sense. Read this first, before 10.2.

READ 10.2 — Git Objects

Day 3 · 60 min

The core of your entire project. Covers blobs, trees, commits, the "type size\data" format, SHA-1 hashing, and how objects are stored under `.git/objects/xx/yyyy...`. **Run every git hash-object and git cat-file command in the book while you read.** Open your own `.git/objects` and look at what is physically there. This subchapter alone covers 80% of what you need to understand. Your Days 4, 5, 6, and 7 implement what this chapter explains.

READ 10.3 — Git References

Day 3 · 30 min

Branches, HEAD (symbolic ref vs detached), lightweight tags, remote-tracking refs (`refs/remotes/origin/main`). The key insight: a branch is literally a 41-byte text file containing a commit hash. Creating one is writing one file. Once that clicks, branching stops being mysterious. Directly maps to your `refs.ts` on Day 9.

SKIM 10.4 — Packfiles

After Day 14 · 30 min

How git bundles loose objects into a single binary packfile using delta compression (storing similar objects as diffs against each other rather than full copies). You are not implementing this. Read it to understand *why it exists*: without packfiles a repo with 10,000 commits would be huge. Interview answer: "Git uses delta compression in packfiles so similar objects are stored as diffs, not full copies — that is how a large repo stays small."

SKIM 10.5 — The Refspec

After Day 9 · 15 min

How remote-tracking refs work — the `refs/remotes/origin/main` naming convention and the mapping between local and remote branch names. You implement a simplified version of this in your pull command. Read it to understand why the naming convention is what it is.

SKIM 10.6 — Transfer Protocols

After Day 16 · 20 min

Dumb HTTP (static file serving, mostly retired) and smart HTTP (want/have/done negotiation over pkt-line binary format). Read it to understand what you chose *not* to implement and why your custom JSON protocol is a valid simplification of the same idea. Skip the pkt-line binary format details — those are wire encoding, not concepts.

SKIP 10.7 — Maintenance and Data Recovery

SKIP 10.8 — Environment Variables

READ 10.9 — Summary

After Day 14 · 5 min

A short wrap-up of the chapter. Read it after finishing Week 2 — by then you will have implemented most of what the chapter covers, and the summary will feel satisfying rather than abstract.

When	Subchapter	Time	What you get
Day 3 (before any code)	10.1 + 10.2 + 10.3	2 hrs	The complete mental model for your project
After Day 9 (refs done)	10.5 The Refspec	15 min	Why remote refs are named the way they are
After Day 14 (CLI done)	10.4 Packfiles + 10.9 Summary	35 min	What you didn't implement and why it exists
After Day 16 (push/pull done)	10.6 Transfer Protocols	20 min	The real protocol vs your custom protocol

How to read 10.2 (the important one): Open two terminals side by side. Left: the Pro Git book. Right: a fresh `git init` repo in `/tmp/test`. Every time the book runs a `git hash-object` or `git cat-file` command, run it yourself and compare the output. Then open `.git/objects/` and find the file that was just created. Reading without doing is half as effective.

Week 1 — TypeScript + Git Object Model

Days 1–7

Goal: By Day 7 you can explain blob, tree, and commit without notes, and `hashObject()` produces the same hash as `sha1sum` in your terminal. TypeScript will feel awkward until Day 6 — that is normal.

☐ **Day 1 TypeScript crash course + project setup** 1.5 hrs

Read TypeScript handbook: Types, Interfaces, Type Aliases, Generics, Modules only. Skip everything else. Read Node.js Buffer docs for 20 min. Then: create `mygit-ts/`, write `package.json` and `tsconfig.json`, install `tsx` and `@types/node`. Write `src/main.ts` that prints “hello mygit”.

Test: `npm tsx src/main.ts` prints without errors.

☐ **Day 2 utils.ts — SHA-1 and zlib** 1.5 hrs

Write `sha1(data: Buffer | string): string` using `node:crypto`. Write `compress(data: Buffer): Buffer` and `decompress(data: Buffer): Buffer` using `node:zlib`.

Test: Hash “blob 5\0hello” and compare with `printf “blob 5\0hello” | sha1sum`. They must match exactly.

☐ **Day 3 Reading day — Git Internals (no code)** 1.5 hrs

Read Pro Git “Git Internals” chapter at git-scm.com/book. Draw on paper: a commit node pointing to a tree node, the tree pointing to two blob nodes. Label every arrow. Write one sentence answering each: What is a blob? What is a tree? What is a commit? What is a ref?

Test: You can draw the blob-tree-commit diagram from memory without looking at your notes.

☐ **Day 4 object.ts part 1 — types + hashing** 1.5 hrs

Define `ObjType`, `TreeEntry`, `GitObject` interfaces. Write `storeBytes(obj): Buffer`: returns “type size\0data” as a Buffer. Write `hashObject(obj): string`: returns SHA-1 of `storeBytes`. Write `makeBlob(content): GitObject`: returns a blob object.

Test: Create a blob with content “hello\n”, call `hashObject`, print the result. Should be a 40-char hex string.

☐ **Day 5 object.ts part 2 — makeCommit and makeTree** 1.5 hrs

Write `makeCommit(treeHash, parentHash, name, email, message): string`: builds the text block with tree, optional parent, author, committer, blank line, message. Write `makeTree(entries): Buffer`: binary format — for each entry, append “mode name\0” then 20 raw bytes from `Buffer.from(hash, 'hex')`.

Test: Print a raw `makeCommit` result with `.toString()`. It should look identical to a real git commit file.

☐ **Day 6 object.ts part 3 — parsers** 1.5 hrs

Write `parseTree(obj): string`: walk `obj.data` byte by byte, find null bytes, extract mode/name per entry, read 20 binary bytes for hash, convert to hex. Write `parseCommit(obj): string`: split on `\n`, read header lines until blank line, collect tree/parent/author, join the rest as message.

Test: Round-trip test: `makeTree([...]) → parseTree → verify entries match`. Same for commit.

☐ **Day 7 * store.ts — content-addressed storage** Weekend · 2 hrs

Write `objectPath(hash): string`: `.mygit/objects/xx/yyyy...` (first 2 chars = subdir). Write `has(hash): boolean`: `fs.existsSync`. Write `write(obj): void`: compute hash, skip if exists (dedup!), compress with `zlib`, `mkdirSync` parent, `writeFileSync`. Write `read(hash): string`: read file, decompress, find null byte, parse type and data.

Test: Write a blob, read it back with a new Store instance. `obj.data.toString()` must equal original content. Open the written file with `xxd` — first bytes should be `78 9c` (zlib magic).

Week 2 — Index, Refs, Repository, First Commands

Days 8–14

Goal: By Day 14 you can run `mygit init`, `add`, `commit`, `log`, `status`, `branch`, and `checkout` — and use `mygit` to version-control its own source code. This is your first complete working artifact.

☐ **Day 8** `index.ts` — the staging area

1.5 hrs

Write `Index` class. `load()`: read `.mygit/index` text file, parse lines "hash path" into `Map<string, string>`. `save()`: write map back, one line per entry, sorted. `stage(path, hash)`, `remove(path)`, `has(path)`, `getHash(path)`, `all()`.

Test: Stage two fake entries, save, create new `Index`, load, confirm both entries present.

☐ **Day 9** `refs.ts` — branches and HEAD

1.5 hrs

Write `Refs` class with private `read(path)` and `write(path, content)`. `headBranch()`: read HEAD, if starts with "ref: refs/heads/" return the branch name, else null. `headCommit()`: follow symbolic ref to actual hash. `setHeadBranch`, `setHeadDetached`, `readBranch`, `writeBranch`, `listBranches`.

Test: Manually create `.mygit/HEAD` containing "ref: refs/heads/main". Call `headBranch()` — must return "main".

☐ **Day 10** `repo.ts` part 1 — init and open

1.5 hrs

Write `Repository` constructor (creates `Store`, `Index`, `Refs`). Write `Repository.init(path)`: create `.mygit/`, `objects/`, `refs/heads/`, write HEAD and config files. Write `Repository.open()`: start at `process.cwd()`, walk toward root, return first dir containing `.mygit`, throw if not found.

Test: Call `Repository.init('/tmp/test')`, cd into it, call `Repository.open()` from a subdirectory — it should find the repo.

☐ **Day 11** `repo.ts` part 2 — `writeTree`

1.5 hrs

Draw on paper first (10 min) before typing. Write `buildTree(entries, prefix)`: iterate the index map, split entries into direct children (no slash after prefix) and subdirectory groups. For each subdir, recurse with extended prefix. Sort all entries by name. Write the tree object. Return its hash. Write `writeTree()`: calls `buildTree(index.all(), "")`.

Test: Stage "src/main.ts" and "README.md". Call `writeTree()`. Read back the root tree — it should have one blob entry and one sub-tree entry for `src`.

☐ **Day 12** `repo.ts` part 3 — log, restore, reachable

1.5 hrs

Write `getLog(startHash?)`: loop — read commit, push to array, follow parent hash, stop when empty. Write `treeToEntries(treeHash, prefix)`: flatten tree recursively into `Map<path, hash>`. Write `restoreTree(treeHash, dir)`: for each tree entry, write blob to disk or recurse into sub-tree. Write `reachable(startHash, stop)`: DFS graph walk collecting all object hashes.

Test: Manually write a commit chain of 3 commits. Call `getLog()` — must return 3 items newest-first.

☐ **Day 13** `init` + `add` + `commit` + `log`

1.5 hrs

Write `commands/init.ts`, `commands/add.ts`, `commands/commit.ts`, `commands/log.ts`. Wire all four into `main.ts` dispatch switch. `add.ts`: read file, write blob, `index.stage()`, `index.save()`. `commit.ts`: `writeTree()`, `makeCommit()`, `store.write()`, advance branch ref.

Test: `mygit init` → create a file → `mygit add` → `mygit commit -m "first"` → `mygit log` shows the commit.

☐ **Day 14 *** `status` + `branch` + `checkout` + integration

Weekend · 3 hrs

Write `commands/status.ts`: (1) staged section: compare index vs HEAD tree, (2) unstaged section: hash working file vs index entry, (3) untracked section: walk working dir, skip anything in index. Write `commands/branch.ts`: no args = list, one arg = create at HEAD. Write `commands/checkout.ts`: delete tracked files, restore new tree, rebuild index, update HEAD.

Test: Full scenario: `init` → `add` → `commit` → `branch` feature → `checkout` feature → `add` new file → `commit` → `checkout` main → new file is GONE. Then: use `mygit` to commit its own source code.

Week 3 — Networking + Backend API

Days 15–21

Goal: By Day 21 you can push a project to a deployed server, browse its files via a REST API, and view commit history. The compare-and-swap on push is the most important thing to understand this week — it is where “non-fast-forward, fetch first” comes from.

☐ **Day 15** server.ts — push and pull endpoints 1.5 hrs

Write the HTTP server using `node:http`. Add `readBody(req)` and `json(res, data)` helpers. Implement **GET** `/refs?name=branch`: read branch hash, return as plain text. Implement **POST** `/push`: parse `{ref, old, new, objects[]}`, compare-and-swap (reject if `current !== old`), write objects, advance ref.

Test: `curl localhost:3000/refs?name=main` returns empty string. `curl -X POST localhost:3000/push -d '{...}'` returns `{"ok": true}`.

☐ **Day 16** pull endpoint + push/pull/clone commands 1.5 hrs

Implement **POST** `/pull`: parse `{ref, have}`, build stop set from have, call `reachable(tip, stop)`, return base64-encoded objects. Write `commands/push.ts`: fetch remote ref, compute missing set, POST objects + new ref. Write `commands/pull.ts`: send have list, write received objects, update tracking ref. Write `commands/clone.ts`: init + pull with empty have + `restoreTree`.

Test: Start server. Push a project. Clone into a fresh directory. All files identical. Run `mygit log` in clone — same history.

☐ **Day 17 *** backend setup + auth Weekend · 3 hrs

Create `backend/`. Install: `express`, `prisma`, `@prisma/client`, `bcrypt`, `jsonwebtoken`, `zod`, type packages. Write `schema.prisma` with User and Repository models. Run `prisma migrate dev`. Implement **POST** `/api/auth/register` (hash with `bcrypt`, save user). Implement **POST** `/api/auth/login` (compare hash, sign JWT). Write auth middleware (verify JWT, attach `req.user`).

Test: Register a user, login, receive JWT. Call a protected route with and without token. 401 without token. 200 with valid token.

☐ **Day 18** repos API + file tree API 1.5 hrs

POST `/api/repos`: create repo in Postgres + call `Repository.init()` on disk (e.g. `repos/{username}/{reponame}/`). **GET** `/api/repos`: list user's repos. **GET** `/api/repos/:owner/:name/tree/:hash`: read tree from object store, call `parseTree()`, return JSON array of entries. **GET** `/api/repos/:owner/:name/blob/:hash`: return raw blob data. **GET** `/api/repos/:owner/:name/root`: return HEAD's root tree hash.

Test: Create a repo via API, push a project via `mygit CLI`. `GET /root` returns a tree hash. `GET /tree/{hash}` returns file entries.

☐ **Day 19** commits API + integration test 1.5 hrs

GET `/api/repos/:owner/:name/commits`: call `getLog()`, return array of `{hash, author, message, parentHash}`. **GET** `/api/repos/:owner/:name/commits/:hash`: return full commit detail. Integration test: push 3 commits via CLI, call commits API, verify 3 commits returned in correct order.

Test: Full read path works: `root` → `tree` → `blob` → `commits`. All via HTTP without touching the filesystem directly.

☐ **Day 20** error handling + Railway deploy 1.5 hrs

Add zod validation to all POST request bodies. Write global error handler middleware (catch-all, return `{error: string}`). Write `Dockerfile`. Deploy to Railway: connect GitHub repo, set env vars (`DATABASE_URL`, `JWT_SECRET`). Run `prisma migrate deploy` in Railway. Test all endpoints on the live `.up.railway.app` URL.

Test: `POST /api/auth/register` works on live Railway URL. Bad request body returns `{error: "...", not a 500}`.

☐ **Day 21** buffer + fix + API docs 1.5 hrs

Fix any bugs found from live testing. Push a real project to the deployed server via `mygit push`. Confirm the

commits API returns real history. Write **API.md** with every endpoint, example **curl** command, and example response.

Test: A colleague (or yourself from a different machine) can push and read files using only the live Railway URL.

Week 4 — Frontend + RAG Pipeline

Days 22–28

Goal: By Day 28 you have a live URL where you can browse a repo's files, view a file with syntax highlighting, see commit history, and ask the codebase a natural language question and get an answer with source file references. That is the complete product.

☐ **Day 22 * Frontend setup + auth**

Weekend · 2.5 hrs

npm create vite@latest frontend - -template react-ts. Install Tailwind CSS, React Router, Axios, date-fns. Configure Vite proxy to backend (avoids CORS in dev). Set up routes: `/login`, `/register`, `/` (dashboard), `/:owner/:repo` (repo page), `/:owner/:repo/commits`. Build login + register pages with form submission. `AuthContext` storing JWT in `localStorage`. Axios request interceptor attaches `Authorization: Bearer` header. `PrivateRoute` redirects to `/login` if not authenticated.

Test: Login with a real account. Verify JWT is stored. Reload page — still logged in. Navigate to a private route without logging in — redirected to `/login`.

☐ **Day 23 Dashboard + repo creation**

1.5 hrs

Dashboard: fetch GET `/api/repos`, render repo cards (name, language chip, last pushed date). “New repository” button navigates to `/new`. Create repo page: name field + submit → POST `/api/repos` → navigate to `/:owner/:reponame` on success.

Test: Create a new repo via the UI. It appears in the dashboard list. Clicking it navigates to the repo page (blank for now).

☐ **Day 24 File browser + folder navigation**

1.5 hrs

Repo home page: fetch GET `/root` for root tree hash, then GET `/tree/:hash` for entries. Render a table: folder icon for 040000 entries, file icon for 100644. Clicking a folder: fetch that sub-tree hash and re-render. Build breadcrumb: `repo / src / commands` — each segment is clickable. Update the URL with the current path so the browser back button works.

Test: Navigate into `src/commands/`, click back — returns to `src/`. Breadcrumb shows the full path at all times.

☐ **Day 25 File viewer + commit history + Vercel deploy**

1.5 hrs

Clicking a file: fetch GET `/blob/:hash`, render content in `<pre>` block with Prism.js syntax highlighting (npm install prismjs). Show file size and line count above. Commit history page: fetch GET `/commits`, render list — short hash (monospace), author, message, relative time with `date-fns`. Deploy to Vercel: set `VITE_API_URL` env var to Railway backend URL.

Test: Browse to a TypeScript file on the live Vercel URL. Keywords like `const` and `async` should be syntax-highlighted. Commit list shows real commits from the pushed project.

☐ **Day 26 * RAG: pgvector setup + schema**

Weekend · 1.5 hrs

In your Railway PostgreSQL database, run: `CREATE EXTENSION IF NOT EXISTS vector;`. Add to `schema.prisma`: `CodeChunk` (id, repoId, filePath, blobHash, chunkIndex, content, startLine) and `CodeEmbedding` (chunkId, embedding vector(1536)). Run `prisma migrate dev`. Get your OpenAI API key. Install `openai` package. Test a raw pgvector query in psql: `SELECT 1 FROM code_embeddings LIMIT 1;` to confirm the table exists.

Test: Migration runs without errors. `CREATE EXTENSION vector` does not throw. pgvector is ready.

☐ **Day 27 * RAG: indexing pipeline**

Weekend · 3 hrs

Write `chunkFile(content, filePath)`: split into 50-line windows (with 10-line overlap), return `{content, startLine, chunkIndex}[]`. Write `embedChunks(chunks)`: call `openai.embeddings.create({model: "text-embedding-3-small"})` in batches of 20 (rate limit protection). Store each embedding in pgvector. Write `deleteStaleChunks(repoId, filePath, oldBlobHash)`: delete `CodeChunk` rows where `blobHash = oldBlobHash`. Wire into push handler: after writing objects, extract blobs (skip trees/commits), delete stale chunks, re-chunk and re-embed. Run this async so push response is not delayed.

Test: Push a 10-file TypeScript project. `SELECT COUNT(*) FROM code_chunks;` returns a non-zero number. Push again

with one file changed — chunk count for that file updates.



Day 28 RAG: retrieval + LLM + Ask tab

1.5 hrs

Write `retrieve(repoId, question, k=5)`: embed question, run `SELECT ... ORDER BY embedding <=> $query_vec LIMIT $k` filtered by `repoId`. Return chunks with `filePath` and `startLine`. Write `synthesise(question, chunks)`: build prompt (system: you are a code expert, each chunk labelled with `file+line`, cite sources in answer), call `gpt-4o`. Write `POST /api/repos/:owner/:name/ask` returning `{answer, sources: {filePath, startLine}[]}`. Add Ask tab to repo page: text input, Send button, render answer, render source chips that navigate to file viewer.

Test: Push mygit's own source code. Ask: "How does writeTree work?" Answer should mention `buildTree` and cite `src/repo.ts`.

Days 29–30 — Polish + Demo

Final stretch

☐ Day 29 Polish + loading states + README

Weekend · 3 hrs

Walk every page on the live Vercel URL. Add loading spinners where data is being fetched. Add empty states: “No commits yet”, “No files in this repo”. Add error states: show an error card when an API call fails. Write README.md sections: what it is, architecture diagram (draw.io), how the RAG pipeline works (chunking → embedding → retrieval → synthesis), tech stack table, live URL.

Test: Every page handles loading, empty, and error states without crashing. README explains the project to someone who has never seen it.

☐ Day 30 Record demo + resume update

Weekend · 2 hrs

Record a 3-minute screen capture showing: (1) Push a project via `mygit push` in the terminal. (2) Navigate to the live URL, browse the file tree. (3) Open a TypeScript file — syntax highlighted. (4) View commit history — real commits. (5) Click the Ask tab, type “How does writeTree work?”, show the answer with source file references. Upload video to YouTube (unlisted) or Loom. Update resume with the project entry.

Test: The demo video is watchable by someone with no context and clearly shows all five features working on the live URL.

You are done when:

- The live URL is accessible and all pages work
- The Ask tab answers questions about a real codebase with source references
- The README explains the project clearly
- The demo video covers all 5 features
- You can explain `writeTree`, `reachable()`, and the RAG pipeline out loud without looking at any notes

Resources

Topic	Resource	When
TypeScript basics	typescriptlang.org/docs/handbook	Day 1 only
Node.js Buffer	nodejs.org/api/buffer.html	Days 1–6 reference
Git Internals	git-scm.com/book (Internals chapter)	Day 3 reading
pgvector	github.com/pgvector/pgvector	Day 26 setup
OpenAI embeddings	platform.openai.com/docs/guides/embeddings	Days 26–27
Prism.js	prismjs.com	Day 25
Railway deploy	docs.railway.app	Day 20
Vercel deploy	vercel.com/docs	Day 25

Interview Prep — 5 Questions You Will Be Asked

Q: What is content-addressed storage?

Files are stored under the SHA-1 hash of their own contents. Same content → same key → stored once automatically. If a single byte changes, the hash changes, so corruption is detectable. This is what gives git its deduplication and integrity guarantees for free.

Q: Walk me through what happens when you run `mygit commit`.

(1) `writeTree` converts the flat index map into nested tree objects recursively, grouping entries by directory. (2) A commit object is created pointing to that root tree and to the parent commit hash. (3) The commit is written to the object store and gets its hash. (4) The current branch ref file is updated to hold the new commit hash. `HEAD` never moves — the branch it points to moves.

Q: How does push know what objects to send?

It performs a graph walk — `reachable(localTip)` minus `reachable(remoteTip)`. Starting from the local branch's commit, it follows every parent and every tree and blob, collecting all hashes. Then it subtracts the set reachable from the remote's current tip. Only the difference travels over the wire. This is why pushing one new commit only sends that commit's objects, not the entire history.

Q: What does checkout do?

It updates three things in sync. (1) Working directory: delete every file tracked on the old branch, restore all files from the new branch's tree object. (2) Index: replace all entries with the new branch's tree entries. (3) `HEAD`: update to point to the new branch. All three must be consistent — if any gets out of sync you get confusing behaviour like files showing as modified when they have not changed.

Q: How does the RAG pipeline work and how do you handle stale embeddings?

On every push, each new blob is chunked into 50-line windows. Each chunk is embedded with `text-embedding-3-small` and stored in pgvector, tagged with its blob hash and file path. At query time, the question is embedded with the same model, and pgvector's cosine similarity operator finds the 5 most relevant chunks. Those chunks plus the question are passed to GPT-4o, which synthesises an answer and cites source files and line numbers. Stale embeddings: when a file changes on a new push, all chunks for the old blob hash are deleted before inserting the new ones. The blob hash is the version key.

The one-sentence project pitch:

"I built git from scratch in TypeScript — content-addressed object store, custom push/pull protocol, GitHub-like React UI — then extended it with a RAG pipeline that embeds code on every push and lets you ask the codebase questions in natural language. I used it to version-control its own development."